

TROC24_8234_ISA

Tagged Register Oriented Computer

p2 mnemonic prefix indicates a two byte, 16-bit instruction

?R column indicates if residue register updated

I suffix indicates immediate value of -6 to +7 or -2028 to +2047

little n to indicate the 5-bit N value in the instruction (-15 to +1

TROC24 16-bit instructions, byte addressing, N indicates 16-bit value in next memory h

mnemonic	encoding	R?	operation
p2IN	000001	N	Store value of input port n into D
p2OUT	001001	N	Place D into output port n
p2LD	000001	N	When n is "10000" load D from memory location N
p2ST	001001	N	When n is "10000" store D into memory location N
p2BSR	010001	N	Branch to PC + N, save address of next inst in D
p2MOV	011001	N	Move register S to D
p2BRZ	100001	N	Branch to PC + N if D is zero
p2BRNZ	101001	N	Branch to PC + N if D is non-zero
P2BRP	110001	N	Branch to PC + N if D is positive (MSB is zero)
P2BRM	111001	N	Branch to PC + N if D is negative (MSB is one)
p2SHFT	000011	Y	if n not "10000" shift or adjust exponent by signed n
p2INSRT	000011	N	if n is "10000" insert field from residue into D per N
p2FUNC	001011		if n not "10000" do a one operand function $fn(D) \Rightarrow D$
p2EXTRCT	001011	~Y	if n is "10000" extract field from D and place in residue
p2AOBR	010011	N	Increment D and branch to N if D not zero
p2SOBR	011011	N	Decrement D and branch to N if D not zero
p2LDIU	100011	N	load unsigned immediate, set tag bits to unsigned
p2LDIS	101011	N	load signed data immediate, set tag bits to signed
p2LDIF	110011	N	load float data immediate, convert N to register format
p2LDIF2	111011	N	load float2 data immediate, convert N to register format
p2ADD	000101	Y	Add S to D
p2SUB	001101	Y	Subtract D from S
p2MUL	010101	Y	Multiply D by S and store in D
P2DIV	011101	Y	Divide D by S and store in D
p2AND	100101	N	Logical AND between S and D to D
p2OR	101101	N	Logical OR between S and D to D
p2XOR	110101	N	Logical exclusive or between S and D to D
p2CMP	111101	~Y	S subtracted from D and placed in residue register
p2ADDI	000111	Y	Add I to D
p2SUBI	001111	Y	Subtract D from I
p2MULI	010111	Y	Multiply D by I and store in D
p2DIVI	011111	Y	Divide D by I and store in D
p2ANDI	100111	N	Logical AND between I and D to D
p2ORI	101111	N	Logical OR between I and D to D
p2XORI	110111	N	Logical exclusive or between I and D to D
p2CMPI	111111	~Y	D subtracted from I and placed in residue register

TROC16 16-bit FUNC instructions

mnemonic	encoding	R?	operation
p2fSETUS	00000	N	set D to unsigned
p2fSETS	00001	N	set D to signed
p2fSETFLT	00010	N	set D to float
p2fSETFLT2	00011	N	set D to float2
p2fABS	00100		take the absolute value of D
p2fNABS	00101		take the negative absolute value of D
p2fTAG	00110	Y	place D's four tag bits into residue
p2fITAG	00111	N	place residue four LSB bits into D's tag
p2fLDUS	01100		Load residue with unsigned from M(D)
p2fLDUS	01101		Load residue with signed from M(D)
p2fLDUS	01110		Load residue with float from M(D)
p2fLDUS	01111		Load residue with float2 from M(D)
p2fST	10001	~Y	convert and store Residue to M(D)
p2fCVTUS	11100	~Y	convert D to unsigned
p2fCVTS	11101	~Y	convert D to signed
p2fCVTFLT	11110	~Y	convert D to float
p2fCVTFLT2	11111	~Y	convert D to float2
			Room for 14 more functions

TROC24 8-bit instructions

mnemonic	encoding	R?	operation
p1TRAP	00000000	N	To catch a wayward instruction pointer
p1BKPT	01000000	N	Take a breakpoint trap
p1ALGN	10000000	N	Force program counter to next word address
p1ALGNC	11000000	N	Force program counter to next cache line address

TROC24 24-bit instructions

mnemonic	encoding	R?	operation
p3LDU8	10 000010	N	Load byte unsigned
p3LDU16	10 000110	N	Load half-word unsigned
p3LDU24	10 001010	N	Load three byte unsigned
p3LDU32	10 001110	N	Load three byte unsigned and its tag
p3LDS8	10 010010	N	Load byte signed
p3LDS16	10 010110	N	Load half-word signed
p3LDS24	10 011010	N	Load three byte signed
p3LDS32	10 011110	N	Load three byte signed and its tag
p3LDF8	10 100010	N	Load byte float
p3LDF16	10 100110	N	Load half-word float
p3LDF24	10 101010	N	Load three byte float
p3LDF32	10 101110	N	Load three byte float and its tag
p3LDSF28	10 110010	N	Load byte float2
p3LDF216	10 110110	N	Load half-word float2
p3LDF224	10 111010	N	Load three byte float2
p3LDF232	10 111110	N	Load three byte float2 and its tag

p3ST16	01 000010	N	Store byte
p3ST16	01 000110	N	Store half-word
p3ST16	01 001010	N	Store three byte
p3ST32	01 001110	N	Store three byte and its tag
p3	01 010010		
p3	01 010110		
p3	01 011010		
p3	01 011110		
p3ADD	01 100010	Y	Add S to D
p3SUB	01 100110	Y	Subtract D from S
p3MUL	01 101010	Y	Multiply D by S and store in D
P3DIV	01 101110	Y	Divide D by S and store in D
p3AND	01 110010	N	Logical AND between S and D to D
p3OR	01 110110	N	Logical OR between S and D to D
p3XOR	01 111010	N	Logical exclusive or between S and D to D
p3CMP	01 111110	~Y	S subtracted from D and placed in residue register
p3INSRT	00 000010		
p3EXTRCT	00 000110		
p3SHL	00 001010		
p3MAX	00 001110		
p3MIN	00 010010		
p3FUNC	00 010110		
p3INTP-BLE	00 011010		
p3CMOV	00 011110		
p3JMPcc	00 100010		
p3BRcc	00 100110		
p3BBS0	00 101010		branch bit set
p3BBS1	00 101110		branch bit set for bits 32 and up
p3BBC0	00 110010		branch bit clear
p3BBC1	00 110110		branch bit clear for bits 32 and up
p3BRccRC	00 111010		
p3CASE	00 111110		

TROC24 32-bit instructions

mnemonic	encoding	R?	operation
p4LDU8	10 000000	N	Load byte unsigned
p4LDU16	10 000100	N	Load half-word unsigned
p4LDU24	10 001000	N	Load three byte unsigned
p4LDU32	10 001100	N	Load three byte unsigned and its tag
p4LDS8	10 010000	N	Load byte signed
p4LDS16	10 010100	N	Load half-word signed
p4LDS24	10 011000	N	Load three byte signed
p4LDS32	10 011100	N	Load three byte signed and its tag
p4LDF8	10 100000	N	Load byte float
p4LDF16	10 100100	N	Load half-word float

p4LDF24	10 101000	N	Load three byte float
p4LDF32	10 101100	N	Load three byte float and its tag
p4LDSF28	10 110000	N	Load byte float2
p4LDF216	10 110100	N	Load half-word float2
p4LDF224	10 111000	N	Load three byte float2
p4LDF232	10 111100	N	Load three byte float2 and its tag
p4ST8	01 000000	N	Store byte to $S + R * gs + T, I$
p4ST16	01 000100	N	Store half-word to $S + 2 * R * gs + T, I$
p4ST24	01 001000	N	Store three byte to $S + 3 * R * gs + T, I$
p4ST32	01 001100	N	Store three byte and its tag to $S + 4 * R * gs + T, I$
p4MISC	01 010000		
p4P3OPS	01 010100		p3 operations except ld/st: $opS(T, R) \Rightarrow D$
p4	01 011000		
p4	01 011100		
p4ADD	01 100000	Y	Add S to D
p4SUB	01 100100	Y	Subtract D from S
p4MAC	01 101000	Y	Multiply R by T add S and store in D
p4DIV	01 101100	Y	Divide R-S by T and store in D
p4AND	01 110000	N	Logical AND between S and D to D
p4OR	01 110100	N	Logical OR between S and D to D
p4XOR	01 111000	N	Logical exclusive or between S and D to D
p4CMP	01 111100	~Y	S subtracted from D and placed in residue register
p1	00 000000		
p4INSRT	00 000100		
p4SHL	00 001000		
p4MAX	00 001100		
p4MIN	00 010000		
p4MEDIAN	00 010100		
p4INTP	00 011000		
p4CMOV	00 011100		
p4JMPcc	00 100000		
p4BRcc	00 100100		
p4	00 101000		
p4	00 101100		
p4VECT	00 110000		
p4LOOP	00 110100		
p4BRccRC	00 111000		
p4CASE	00 111100		
p6	11 xxxx00		48-bit instructions
p5	11 xxxx10		40-bit instructions

using an additional byte or a three or four byte N
5) and big N (little n is “10000”)

half-word

more detail
Negative n signifies internal ports, Positive n external ports.
Negative n signifies internal ports, Positive n external ports.
floating point load does memory to register format conversion
floating point store does register to memory format conversion
tag bits moved as well
N format: uu0000www00mmmm data type, bit width, starting bit
N format: uu0000www00mmmm data type, bit width, starting bit
N is 2's complement
N is 2's complement
N is in memory format
N is in memory format
Floating-point ADD makes use of & may modify the exponent tag bits
Floating-point SUB makes use of & may modify the exponent tag bits
Floating-point MUL makes use of & may modify the exponent tag bits
Floating-point DIV makes use of & may modify the exponent tag bits
operation is irrespective of data types
operation is irrespective of data types
operation is irrespective of data types
Floating-point CMP makes use of & may modify the exponent tag bits
for floating op I converted to register format of D before add
for floating op I converted to register format of D before subtract
for floating op I converted to register format of D before multiply
for floating op I converted to register format of D before divide
operation is irrespective of data types
operation is irrespective of data types
operation is irrespective of data types
for floating op I converted to register format of D before subtract

if n not "10000" do a one operand function $fn(D) \Rightarrow D$

[illegible]

more detail
Illegal or unimplemented instruction trap
for debugging purposes

If R is specified as the PC (register 31) a value of zero is used

more detail
If immediate address is register S + N, otherwise address is S + R
If immediate address is register S + N, otherwise address is S + 2 * R
If immediate address is register S + N, otherwise address is S + 3 * R
If immediate address is register S + N, otherwise address is S + 4 * R
If immediate address is register S + N, otherwise address is S + R
If immediate address is register S + N, otherwise address is S + 2 * R
If immediate address is register S + N, otherwise address is S + 3 * R
If immediate address is register S + N, otherwise address is S + 4 * R
If immediate address is register S + N, otherwise address is S + R
If immediate address is register S + N, otherwise address is S + 2 * R
If immediate address is register S + N, otherwise address is S + 3 * R
If immediate address is register S + N, otherwise address is S + 4 * R
If immediate address is register S + N, otherwise address is S + R
If immediate address is register S + N, otherwise address is S + 2 * R
If immediate address is register S + N, otherwise address is S + 3 * R
If immediate address is register S + N, otherwise address is S + 4 * R

[illegible]

bits 26..24 indicate the number of data grouped items ("gs" from one to eight)

more detail
address is $S + R * gs + T, l$
address is $S + 2 * R * gs + T, l$
address is $S + 3 * R * gs + T, l$
address is $S + 4 * R * gs + T, l$
address is $S + R * gs + T, l$
address is $S + 2 * R * gs + T, l$
address is $S + 3 * R * gs + T, l$
address is $S + 4 * R * gs + T, l$
address is $S + R * gs + T, l$
address is $S + 2 * R * gs + T, l$

address is $S + 3 * R * gs + T, I$
address is $S + 4 * R * gs + T, I$
address is $S + R * gs + T, I$
address is $S + 2 * R * gs + T, I$
address is $S + 3 * R * gs + T, I$
address is $S + 4 * R * gs + T, I$
also p4ST8I store immediate byte
also p4ST16I store immediate half-word
also p4ST24I store immediate three byte
also p4ST32I store immediate word
MMOV, PCND, PCB1, STM, LDM, MUX
p3 instructions with three additional op-code bite
also p4MERGE
TRAP inst
also p4BLEND-LERP, p4FINTP
also p4CALLcc
also p4BSRcc
also p4JMPccRC